Problem Solving

25 more points to get your next star!

Rank: 2033487 | Points: 75/100

Problem Solving

Algorithms | Data Structures

Arrays - DS
Easy, Problem Solving (Basic), Max Score: 10, Success Rate: 93.02%

Solved

2D Array - DS
Easy, Problem Solving (Basic), Max Score: 15, Success Rate: 93.18%

Solved

Left Rotation
Easy, Problem Solving (Basic), Max Score: 20, Success Rate: 91.54%

Solved

Print the Elements of a Linked List
Easy, Problem Solving (Basic), Max Score: 5, Success Rate: 97.14%

Solved

Insert a Node at the Tail of a Linked List
Easy, Problem Solving (Intermediate), Max Score: 5, Success Rate: 95.31%

Solved

Insert a node at the head of a linked list
Easy, Problem Solving (Basic), Max Score: 5, Success Rate: 98.31%

Solved

Insert a node at a specific position in a linked list
Easy, Problem Solving (Intermediate), Max Score: 5, Success Rate: 96.94%

Solved

Delete a Node
Easy, Problem Solving (Basic), Max Score: 5, Success Rate: 95.20%

Solved

Print in Reverse
Easy, Problem Solving (Basic), Max Score: 5, Success Rate: 97.94%

Solved

An array is a type of data structure that stores elements of the same type in a contiguous block of memory. In an array, A, of size N, each memory location has some unique index, i (where $0 \le i < N$), that can be referenced as $A[i]$ or A_i.

Reverse an array of integers.

Note: If you've already solved our C++ domain's Arrays Introduction challenge, you may want to skip this.

Example

$A = [1, 2, 3]$

Return $[3, 2, 1]$.

Function Description

Complete the function reverseArray in the editor below.

reverseArray has the following parameter(s):

- int A[n]: the array to reverse

Returns

- int[n]: the reversed array

Input Format

The first line contains an integer, N, the number of integers in A.

The second line contains N space-separated integers that make up A.

Constraints

- $1 \le N \le 10^3$
- $1 \le A[i] \le 10^4$, where $A[i]$ is the ith integer in A

Sample Input 1

Copy Download

1	4	3	2

Array: arr

```
4
1 4 3 2
```

Sample Output 1

```
2 3 4 1
```

Change Theme Language C

```c
 *     for (int i = 0; i < 5; i++) {
 *         *(a + i) = i + 1;
 *     }
 *
 *     return a;
 * }
 *
 */
int* reverseArray(int a_count, int* a, int* result_count) {
    int* b=malloc(a_count * sizeof(int));
     for (int i = 0; i < a_count; i++) {
         *(b+a_count-1-i)= *(a + i);
     }
     *result_count=a_count;
     return b;
}

int main()
{
    FILE* fptr = fopen(getenv("OUTPUT_PATH"), "w");

    int arr_count = parse_int(ltrim(rtrim(readline())));

    char** arr_temp = split_string(rtrim(readline()));

```

Line: 215 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Test case 0
Test case 1
Test case 2
Test case 3
Test case 4
Test case 5
Test case 6

Compiler Message

Success

Input (stdin) Download

```
8
6676 3216 4063 8373 423 586 8850 6762
```

Expected Output Download

```
6762 8850 586 423 8373 4063 3216 6676
```

Given a 6×6 2D Array, arr:

```
1 1 1 0 0 0
0 1 0 0 0 0
1 1 1 0 0 0
0 0 0 0 0 0
0 0 0 0 0 0
0 0 0 0 0 0
```

An hourglass in A is a subset of values with indices falling in this pattern in arr's graphical representation:

```
a b c
  d
e f g
```

There are 16 hourglasses in arr. An hourglass sum is the sum of an hourglass' values. Calculate the hourglass sum for every hourglass in arr, then print the maximum hourglass sum. The array will always be 6×6.

Example

$arr =$

```
-9 -9 -9  1 1 1
 0 -9  0  4 3 2
-9 -9 -9  1 2 3
 0  0  8  6 6 0
 0  0  0 -2 0 0
 0  0  1  2 4 0
```

The 16 hourglass sums are:

```
-63, -34, -9, 12,
-10,  0, 28, 23,
-27, -11, -2, 10,
  9,  17, 25, 18
```

The highest hourglass sum is 28 from the hourglass beginning at row 1, column 2:

```
0 4 3
  1
8 6 6
```

Note: If you have already solved the Java domain's Java 2D Array challenge, you may wish to skip this challenge.

Function Description

Complete the function hourglassSum in the editor below.

hourglassSum has the following parameter(s):

- int arr[6][6]: an array of integers

Returns

- int: the maximum hourglass sum

Input Format

Each of the 6 lines of inputs $arr[i]$ contains 6 space-separated integers $arr[i][j]$.

Change Theme Language C

```c
/*
 * Complete the 'hourglassSum' function below.
 *
 * The function is expected to return an INTEGER.
 * The function accepts 2D_INTEGER_ARRAY arr as parameter.
 */

int hourglassSum(int arr_rows, int arr_columns, int** arr)
    int i,j,sum,max;
    for (i=0; i<4; i++) {
        for (j=0; j<4; j++) {
            sum= *(*(arr + i) + j)+ *(*(arr + i) + j+1)+ *(
            + *(*(arr + i+1) + j+1)+
     *(*(arr + i+2) + j)+ *(*(arr + i+ 2) + j+1)+ *(*(arr +
            if (i==0&&j==0) {
            max=sum;
            }
            if (sum>max) {
            max=sum;
            }
        }
    }
    return max;
}
```

Line: 190 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Test case 0
Test case 1
Test case 2
Test case 3
Test case 4
Test case 5
Test case 6

Compiler Message

Success

Input (stdin) Download

```
1 1 1 0 0 0
0 1 0 0 0 0
1 1 1 0 0 0
0 0 2 4 4 0
0 0 0 2 0 0
0 0 1 2 4 0
```

Expected Output Download

A left rotation operation on an array of size n shifts each of the array's elements 1 unit to the left. Given an integer, d, rotate the array that many steps left and return the result.

Example

$d = 2$

$arr = [1, 2, 3, 4, 5]$

After 2 rotations, $arr' = [3, 4, 5, 1, 2]$.

Function Description

Complete the rotateLeft function in the editor below.

rotateLeft has the following parameters:

- int d: the amount to rotate by
- int arr[n]: the array to rotate

Returns

- int[n]: the rotated array

Input Format

The first line contains two space-separated integers that denote n, the number of integers, and d, the number of left rotations to perform.

The second line contains n space-separated integers that describe $arr[]$.

Constraints

- $1 \le n \le 10^5$
- $1 \le d \le n$
- $1 \le a[i] \le 10^6$

Sample Input

```
5 4
1 2 3 4 5
```

Sample Output

```
5 1 2 3 4
```

Explanation

To perform $d = 4$ left rotations, the array undergoes the following sequence of changes:

$[1, 2, 3, 4, 5] \rightarrow [2, 3, 4, 5, 1] \rightarrow [3, 4, 5, 1, 2] \rightarrow [4, 5, 1, 2, 3] \rightarrow$

Change Theme Language C

```c
 * }
 *
 */
int* rotateLeft(int d, int arr_count, int* arr, int* result
    int i,j,x;
    for (i=0; i<d; i++) {
        x=*(arr);
        for (j=0; j<arr_count-1; j++) {
            *(arr+j)=*(arr+j+1);
        }
        *(arr+arr_count-1)=x;
    }
    *result_count=arr_count;
    return arr;
}

int main()
{
    FILE* fptr = fopen(getenv("OUTPUT_PATH"), "w");

    char** first_multiple_input = split_string(rtrim(readli

    int n = parse_int(*(first_multiple_input + 0));

    int d = parse_int(*(first_multiple_input + 1));
```

Line: 225 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Test case 0
Test case 1
Test case 2
Test case 3
Test case 4
Test case 5
Test case 6

Compiler Message

Success

Input (stdin) Download

```
5 4
1 2 3 4 5
```

Expected Output Download

```
5 1 2 3 4
```

This challenge is part of a MyCodeSchool tutorial track and is accompanied by a video lesson.

This is an to practice traversing a linked list. Given a pointer to the head node of a linked list, print each node's $data$ element, one per line. If the head pointer is null (indicating the list is empty), there is nothing to print.

Function Description

Complete the printLinkedList function in the editor below.

printLinkedList has the following parameter(s):

- SinglyLinkedListNode head: a reference to the head of the list

Print

- For each node, print its $data$ value on a new line (console.log in Javascript).

Input Format

The first line of input contains n, the number of elements in the linked list. The next n lines contain one element each, the $data$ values for each node.

Note: Do not read any input from stdin/console. Complete the printLinkedList function in the editor below.

Constraints

- $1 \le n \le 1000$
- $1 \le list[i] \le 1000$, where $list[i]$ is the ith element of the linked list.

Sample Input

```
2
16
13
```

Sample Output

```
16
13
```

Explanation

There are two elements in the linked list. They are represented as 16 -> 13 -> NULL. So, the printLinkedList function should print 16 and 13 each on a new line.

Change Theme Language C

```c
#include <assert.h> ...

// Complete the printLinkedList function below.

/*
 * For your reference:
 *
 * SinglyLinkedListNode {
 *     int data;
 *     SinglyLinkedListNode* next;
 * };
 *
 */
void printLinkedList(SinglyLinkedListNode* head) {
    while(head!=NULL){
        printf("%d\n",head->data);
        head=head->next;
    }
}

int main() ...
```

Line: 55 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Test case 0
Test case 1
Test case 2
Test case 3
Test case 4
Test case 5
Test case 6

Compiler Message

Success

Input (stdin) Download

```
2
16
13
```

Expected Output Download

```
16
13
```

This challenge is part of a tutorial track by MyCodeSchool and is accompanied by a video lesson.

You are given the pointer to the head node of a linked list and an integer to add to the list. Create a new node with the given integer. Insert this node at the tail of the linked list and return the head node of the linked list formed after inserting this new node. The given head pointer may be null, meaning that the initial list is empty.

Function Description

Complete the insertNodeAtTail function in the editor below.

insertNodeAtTail has the following parameters:

- SinglyLinkedListNode pointer head: a reference to the head of a list
- int data: the data value for the node to insert

Returns

- SinglyLinkedListNode pointer: reference to the head of the modified linked list

Input Format

The first line contains an integer n, the number of elements in the linked list.

The next n lines contain an integer each, the value that needs to be inserted at tail.

Constraints

- $1 \leq n \leq 1000$
- $1 \leq list_i \leq 1000$

Sample Input

STDIN Function ----- -------- 5 size of linked list n = 5 141 linked list data values 141..474 302 164 530 474

Sample Output

```
141
302
164
530
474
```

Explanation

First the linked list is NULL. After inserting 141, the list is 141 -> NULL. After inserting 302, the list is 141 -> 302 -> NULL.
After inserting 164, the list is 141 -> 302 -> 164 -> NULL.
After inserting 530, the list is 141 -> 302 -> 164 -> 530 -> NULL. After inserting 474, the list is 141 -> 302 -> 164 -> 530 -> 474 -> NULL, which is the final list.

Change Theme Language C

```c
/*
 * For your reference:
 *
 * SinglyLinkedListNode {
 *     int data;
 *     SinglyLinkedListNode* next;
 * };
 *
 */
SinglyLinkedListNode* insertNodeAtTail(SinglyLinkedListNode
    SinglyLinkedListNode* y;
    SinglyLinkedListNode* x= head;
    y= malloc(sizeof(SinglyLinkedListNode));
    y->data=data;
    y->next=NULL;
    if (head==NULL) {
        return y;
    }
    while(x->next!=NULL)
        {x=x->next;}
    x->next=y;
    return head;
}

int main() ···
```

Line: 54 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

Test case 0
Test case 1
Test case 2
Test case 3
Test case 4
Test case 5
Test case 6

Compiler Message

Success

Input (stdin) Download

```
5
141
302
164
530
474
```

Expected Output Download

This challenge is part of a tutorial track by MyCodeSchool and is accompanied by a video lesson.

Given a pointer to the head of a linked list, insert a new node before the head. The ***next*** value in the new node should point to ***head*** and the ***data*** value should be replaced with a given value. Return a reference to the new head of the list. The head pointer given may be null meaning that the initial list is empty.

Function Description

Complete the function insertNodeAtHead in the editor below.

insertNodeAtHead has the following parameter(s):

- SinglyLinkedListNode llist: a reference to the head of a list
- data: the value to insert in the ***data*** field of the new node

Input Format

The first line contains an integer n, the number of elements to be inserted at the head of the list.

The next n lines contain an integer each, the elements to be inserted, one per function call.

Constraints

- $1 \leq n \leq 1000$
- $1 \leq list[i] \leq 1000$

Sample Input

```
5
383
484
392
975
321
```

Sample Output

```
321
975
392
484
383
```

Explanation

Intially the list in NULL. After inserting 383, the list is 383 -> NULL.

After inserting 484, the list is 484 -> 383 -> NULL.

After inserting 392, the list is 392 -> 484 -> 383 -> NULL.

After inserting 975, the list is 975 -> 392 -> 484 -> 383 -> NULL.

After inserting 321, the list is 321 -> 975 -> 392 -> 484 -> 383 -> NULL.

Change Theme Language C

```c
#include <assert.h> ...

// Complete the insertNodeAtHead function below.

/*
 * For your reference:
 *
 * SinglyLinkedListNode {
 *     int data;
 *     SinglyLinkedListNode* next;
 * };
 *
 */
SinglyLinkedListNode* insertNodeAtHead(SinglyLinkedListNode
    SinglyLinkedListNode* x=malloc( sizeof(SinglyLinkedList
    x->data=data;
    x->next=llist;
    return x;

}

int main() ...
```

Line: 55 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

Test case 0
Test case 1
Test case 2
Test case 3
Test case 4
Test case 5
Test case 6

Compiler Message

Success

Input (stdin) Download

```
5
383
484
392
975
321
```

Expected Output Download

```
321
```

This challenge is part of a tutorial track by MyCodeSchool and is accompanied by a video lesson.

Given the pointer to the head node of a linked list and an integer to insert at a certain position, create a new node with the given integer as its $data$ attribute, insert this node at the desired position and return the head node.

A position of 0 indicates head, a position of 1 indicates one node away from the head and so on. The head pointer given may be null meaning that the initial list is empty.

Example

$head$ refers to the first node in the list $1 \rightarrow 2 \rightarrow 3$

$data = 4$

$position = 2$

Insert a node at position 2 with $data = 4$. The new list is

$1 \rightarrow 2 \rightarrow 4 \rightarrow 3$

Function Description Complete the function insertNodeAtPosition in the editor below. It must return a reference to the head node of your finished list.

insertNodeAtPosition has the following parameters:

- head: a SinglyLinkedListNode pointer to the head of the list
- data: an integer value to insert as data in your new node
- position: an integer position to insert the new node, zero based indexing

Returns

- SinglyLinkedListNode pointer: a reference to the head of the revised list

Input Format

The first line contains an integer n, the number of elements in the linked list.

Each of the next n lines contains an integer SinglyLinkedListNode[i].data.

The next line contains an integer $data$, the data of the node that is to be inserted.

The last line contains an integer $position$.

Constraints

- $1 \leq n \leq 1000$
- $1 \leq SinglyLinkedListNode[i].data \leq 1000$, where $SinglyLinkedListNode[i]$ is the ith element of the linked list.
- $0 \leq position \leq n$.

Sample Input

```
3
16
13
7
1
2
```

Sample Output

Change Theme Language C

```c
 *
 * SinglyLinkedListNode {
 *     int data;
 *     SinglyLinkedListNode* next;
 * };
 *
 */

SinglyLinkedListNode* insertNodeAtPosition(SinglyLinkedList
data, int position) {
    SinglyLinkedListNode* x=malloc(sizeof(SinglyLinkedListN
    x->data=data;
    SinglyLinkedListNode* y=llist;
    if (position==0) {
        x->next=llist;
        return x;
    }
    for (int i=0; i<position-1; i++) {
        y=y->next;
    }
    x->next=y->next;
    y->next=x;
    return llist;
}

int main()...
```

Line: 67 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

Test case 0
Test case 1
Test case 2
Test case 3
Test case 4
Test case 5
Test case 6

Compiler Message

Success

Input (stdin) Download

```
3
16
13
7
1
2
```

Expected Output Download

```
16 13 1 7
```

This challenge is part of a tutorial track by MyCodeSchool and is accompanied by a video lesson.

Delete the node at a given position in a linked list and return a reference to the head node. The head is at position 0. The list may be empty after you delete the node. In that case, return a null value.

Example

$llist = 0 \rightarrow 1 \rightarrow 2 \rightarrow 3$

$position = 2$

After removing the node at position 2, $llist' = 0 \rightarrow 1 \rightarrow 3$.

Function Description

Complete the deleteNode function in the editor below.

deleteNode has the following parameters:

- SinglyLinkedListNode pointer llist: a reference to the head node in the list
- int position: the position of the node to remove

Returns

- SinglyLinkedListNode pointer: a reference to the head of the modified list

Input Format

The first line of input contains an integer n, the number of elements in the linked list.

Each of the next n lines contains an integer, the node data values in order.

The last line contains an integer, $position$, the position of the node to delete.

Constraints

- $1 \leq n \leq 1000$
- $1 \leq list[i] \leq 1000$, where $list[i]$ is the ith element of the linked list.

Sample Input

```
8
20
6
2
19
7
4
15
9
3
```

Sample Output

```
20 6 2 7 4 15 9
```

Explanation

The original list is $20 \rightarrow 6 \rightarrow 2 \rightarrow 19 \rightarrow 7 \rightarrow 4 \rightarrow 15 \rightarrow 9$. After deleting the node at position 3, the list is

$20 \rightarrow 6 \rightarrow 2 \rightarrow 7 \rightarrow 4 \rightarrow 15 \rightarrow 9$.

Change Theme Language C

```
 *  2. INTEGER position
 */

/*
 * For your reference:
 *
 * SinglyLinkedListNode {
 *     int data;
 *     SinglyLinkedListNode* next;
 * };
 *
 */

SinglyLinkedListNode* deleteNode(SinglyLinkedListNode* llis
    SinglyLinkedListNode* x= llist;
    if (position==0) {
    return llist->next;
    }
    for (; position!=1; position--) {
    x=x->next;
    }
    x->next=x->next->next;
    return llist;
}

int main()…
```

Line: 67 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

Test case 0
Test case 1
Test case 2
Test case 3
Test case 4
Test case 5
Test case 6

Compiler Message

Success

Input (stdin) Download

```
8
20
6
2
19
7
4
15
9
```

This challenge is part of a tutorial track by MyCodeSchool and is accompanied by a video lesson.

Given a pointer to the head of a singly-linked list, print each $data$ value from the reversed list. If the given list is empty, do not print anything.

Example

$head*$ refers to the linked list with $data$ values

$1 \rightarrow 2 \rightarrow 3 \rightarrow NULL$

Print the following:

```
3
2
1
```

Function Description

Complete the reversePrint function in the editor below.

reversePrint has the following parameters:

- SinglyLinkedListNode pointer head: a reference to the head of the list

Prints

The $data$ values of each node in the reversed list.

Input Format

The first line of input contains t, the number of test cases.

The input of each test case is as follows:

- The first line contains an integer n, the number of elements in the list.
- Each of the next n lines contains a data element for a list node.

Constraints

- $1 \leq n \leq 1000$
- $1 \leq list[i] \leq 1000$, where $list[i]$ is the ith element in the list.

Sample Input

```
3
5
16
12
4
2
5
3
7
3
9
5
5
1
18
3
13
```

Sample Output

```
5
2
```

Change Theme Language C

```
 * Complete the reversePrint function below.
 *
 * The function accepts INTEGER_SINGLY_LINKED_LIST llist as
 */

/*
 * For your reference:
 *
 * SinglyLinkedListNode {
 *     int data;
 *     SinglyLinkedListNode* next;
 * };
 *
 */

void reversePrint(SinglyLinkedListNode* llist) {
    if (llist->next==NULL) {
        printf("%d\n",llist->data);
        return;
    }
    reversePrint(llist->next);
    printf("%d\n",llist->data);
    return;
}

int main()…
```

Line: 67 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

Test case 0
Test case 1
Test case 2
Test case 3
Test case 4
Test case 5
Test case 6

Compiler Message

Success

Input (stdin) Download

```
3
5
16
12
4
2
5
3
7
```

This challenge is part of a tutorial track by MyCodeSchool

Given pointers to the heads of two sorted linked lists, merge them into a single, sorted linked list. Either head pointer may be null meaning that the corresponding list is empty.

Example

$headA$ refers to $1 \rightarrow 3 \rightarrow 7 \rightarrow NULL$

$headB$ refers to $1 \rightarrow 2 \rightarrow NULL$

The new list is $1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 7 \rightarrow NULL$

Function Description

Complete the mergeLists function in the editor below.

mergeLists has the following parameters:

- SinglyLinkedListNode pointer headA: a reference to the head of a list
- SinglyLinkedListNode pointer headB: a reference to the head of a list

Returns

- SinglyLinkedListNode pointer: a reference to the head of the merged list

Input Format

The first line contains an integer t, the number of test cases.

The format for each test case is as follows:

The first line contains an integer n, the length of the first linked list.

The next n lines contain an integer each, the elements of the linked list.

The next line contains an integer m, the length of the second linked list.

The next m lines contain an integer each, the elements of the second linked list.

Constraints

- $1 \leq t \leq 10$
- $1 \leq n, m \leq 1000$
- $1 \leq list[i] \leq 1000$, where $list[i]$ is the ith element of the list.

Sample Input

```
1
3
1
2
3
2
3
4
```

Sample Output

```
1 2 3 3 4
```

Explanation

The first linked list is: $1 \rightarrow 3 \rightarrow 7 \rightarrow NULL$

Change Theme Language C

```c
 *
 * SinglyLinkedListNode {
 *     int data;
 *     SinglyLinkedListNode* next;
 * };
 *
 */
SinglyLinkedListNode* mergeLists(SinglyLinkedListNode* head
SinglyLinkedListNode* head2) {
    if (head2==NULL) {
        return head1;
    } else if (head1==NULL) {
        return head2;
    } else{
        if (head1->data<head2->data) {
            head1->next=mergeLists(head1->next, head2);
            return head1;
        } else {
            head2->next=mergeLists(head1, head2->next);
            return head2;
        }
    }

}

int main()…
```

Line: 67 Col: 1

Upload Code as File | Test against custom input | Run Code | Submit Code

Test case 0
Test case 1
Test case 2
Test case 3
Test case 4
Test case 5
Test case 6

Compiler Message

Success

Input (stdin) Download

```
1
3
1
2
3
2
3
4
```